LAB 6

Sense | Localize | Map | Plan | Track ◀ | Autonomous

BLOCK 4 — FOLLOW THE PATH

Path Tracking & Control

Introduction to Robotics for AI and Data Science

The robot has a plan. The map is built. The path is computed.

Now — how does it actually drive the path?

AF

Dr. Akram Fatayer

a.fatayer@zuj.edu.jo · linkedin.com/in/akram-fatayer

Integration Map: Closing the Loop

Lab 6 is the final piece — the robot can now execute its plan

Sense — Lab 1 — Done

Localize — Lab 3 — Done

Map — Lab 4 — Done

Plan — Lab 5 — Done

Track — Lab 6 — NOW — Current

Autonomous — Final Project

What Lab 5 Gave Us

A* computed a sequence of **waypoints** — discrete (x, y) positions the robot should pass through. This is the global path: optimal, collision-free, but static.

What Lab 6 Adds

A **path tracking controller** that converts the waypoint sequence into real-time **wheel velocity commands**. The robot now physically drives the planned path — smoothly and accurately.

The Missing Link

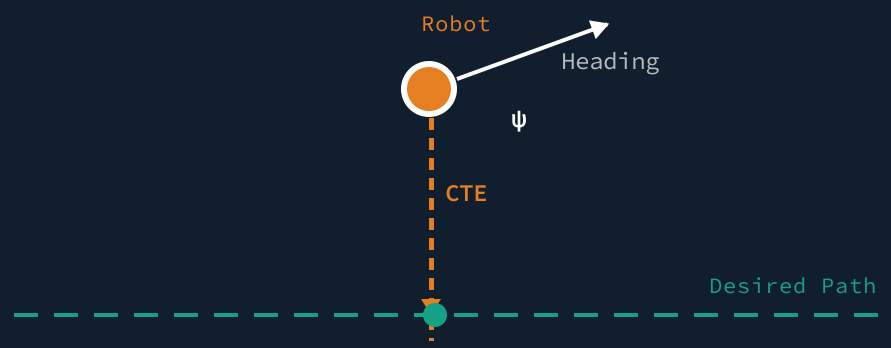
Without a controller, the robot cannot follow a path — it would just stop at each waypoint. The controller is the bridge between planning and motion.

After Lab 6: Your warehouse robot can sense its environment, localize itself, build a map, plan a collision-free path, and now — follow that path to its destination. The autonomy stack is complete.

The Path Tracking Problem

Two error quantities — one controller goal

ERROR GEOMETRY



Cross-Track Error (CTE)

```
e = perpendicular distance to path
```

The **lateral distance** between the robot's current position and the nearest point on the desired path. Positive when robot is to the left, negative to the right. This is what the **P term** in PID corrects.

Heading Error (ψ)

```
ψ = θ_robot − θ_path_tangent
```

The **angular difference** between the robot's current heading and the direction of the path at the nearest point. If $\psi \neq 0$, the robot is pointing away from the path — it will drift further even if CTE is small.

THE CONTROLLER'S SINGLE GOAL

$$CTE \to 0 \quad \text{and} \quad \psi \to 0$$

Every path tracking algorithm — PID, Pure Pursuit, Stanley — is a different strategy to drive these two errors to zero simultaneously.

The Unicycle Model

The kinematic foundation for our tracking controllers

STATE VECTOR

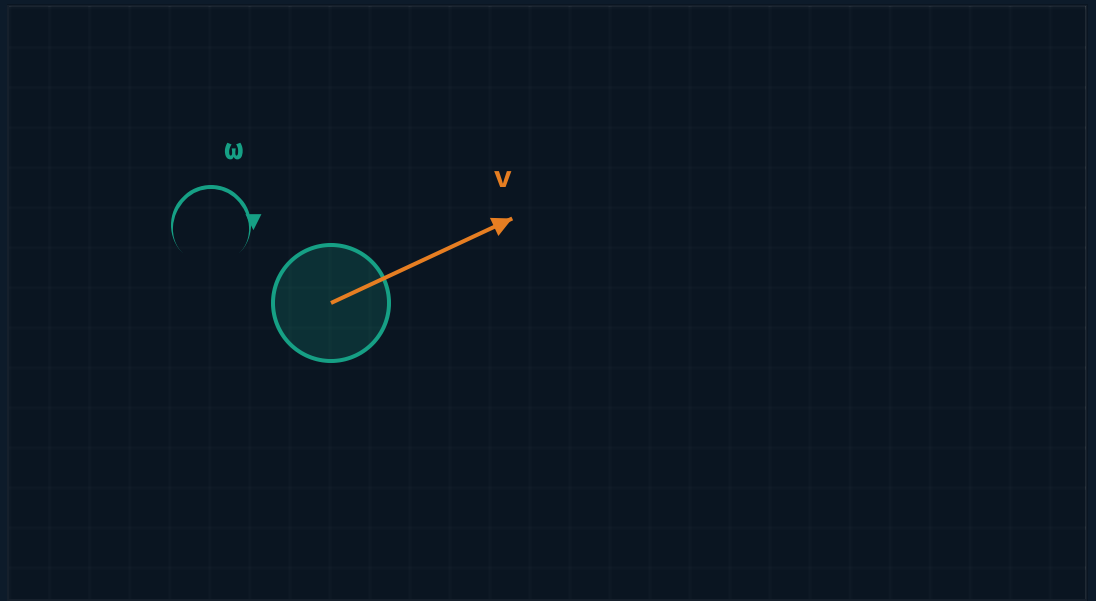
$$[\ x,\ y,\ \theta\ ]^T$$

CONTROL INPUTS

$$[\ v,\ \omega\ ]^T$$

KINEMATIC EQUATIONS

$$\dot{x} = v \cdot \cos(\theta)$$
$$\dot{y} = v \cdot \sin(\theta)$$
$$\dot{\theta} = \omega$$



Why the Unicycle Model?

The unicycle model is the simplest representation of a mobile robot. It captures the essential **non-holonomic constraint**: the robot can only move forward/backward in the direction it is facing, and it can rotate. It cannot move sideways.

By designing our controllers to output linear velocity (**v**) and angular velocity (**ω**), we make the control logic independent of the specific robot hardware.

Mapping to Differential Drive

Our warehouse robot uses a **differential drive** system (two independent wheels). We can easily convert the unicycle commands (v, ω) into left and right wheel velocities (v_L, v_R):

CONVERSION EQUATIONS

$$v_R = v + (\omega \cdot L / 2)$$
$$v_L = v - (\omega \cdot L / 2)$$

Where L is the wheelbase (distance between the wheels).

PID Control: The Universal Baseline

Applying Proportional, Integral, and Derivative control to Cross-Track Error

THE PID CONTROL LAW

$$u(t) = K_p \cdot e(t) + K_i \cdot \int e(t)\,dt + K_d \cdot de(t)/dt$$

For path tracking: **u(t)** = Steering Angle (δ), **e(t)** = Cross-Track Error (CTE)

P Proportional

Steers the robot **proportionally** to the current CTE. The further off the path, the harder it steers back.

Problem: A pure P-controller overshoots the path, causing continuous oscillation.

```
High K_p → Fast response, high oscillation
Low K_p → Sluggish, slow to return
```

I Integral

Sums up the CTE over time. It eliminates **steady-state error** caused by mechanical bias (e.g., misaligned wheels) or environmental factors.

Problem: Can cause "integral windup" if error accumulates too long.

```
High K_i → Fixes bias, but adds instability
Low K_i → Bias remains uncorrected
```

D Derivative

Reacts to the **rate of change** of the CTE. As the robot approaches the path, the D term counter-steers to **dampen the oscillation**.

Problem: Highly sensitive to sensor noise.

```
High K_d → Smooths motion, amplifies noise
Low K_d → Fails to stop P-term oscillation
```

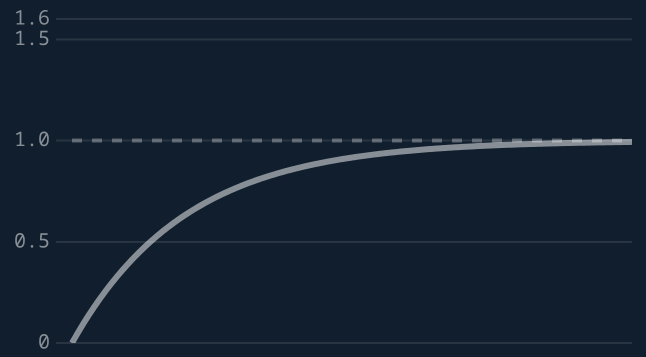
PID Tuning: Understanding the Response

How the P, I, and D terms affect the robot's path tracking behavior

Underdamped (Oscillating)

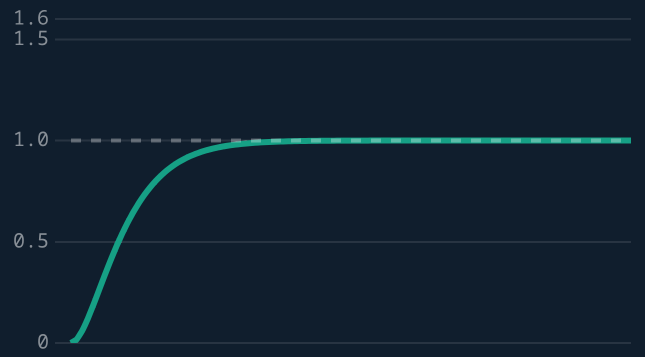
Kp too high or **Kd too low**. The robot overshoots the path and weaves back and forth.

Overdamped (Sluggish)



Kp too low or **Kd too high**. The robot takes too long to converge to the path, cutting corners.

Critically Damped (Ideal)



Well-tuned. The robot quickly converges to the path without overshooting.

THE ZIEGLER-NICHOLS TUNING METHOD

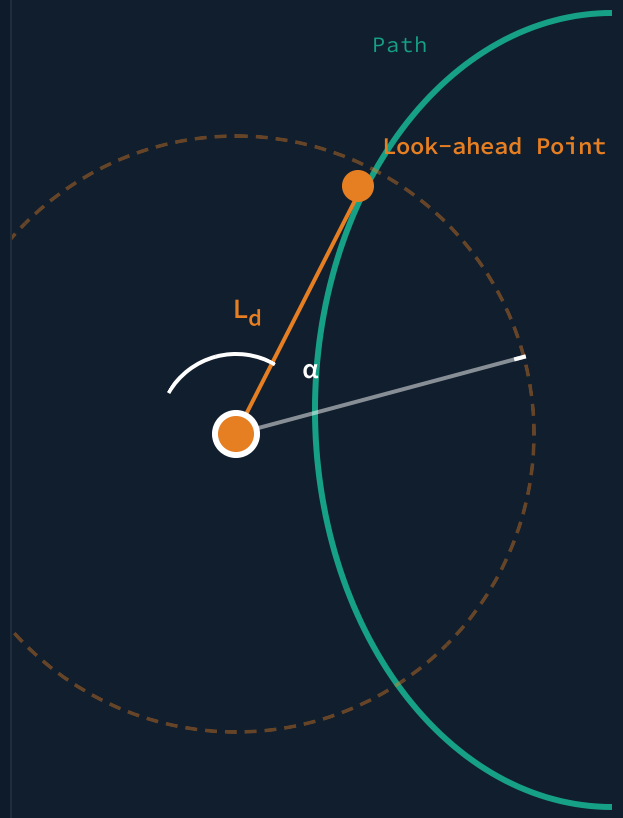
1. Set `Ki = 0` and `Kd = 0`.
2. Increase `Kp` until the robot oscillates steadily (Ultimate Gain, `Ku`).
3. Measure the period of oscillation (`Tu`).
4. Calculate final gains:

```
Kp = 0.6 Ku
Ki = 1.2 Ku / Tu
Kd = 0.075 Ku × Tu
```

Pure Pursuit: Geometry

Following a path by chasing a moving target

PURE PURSUIT GEOMETRY



The Concept

Instead of calculating complex error derivatives like PID, Pure Pursuit uses pure geometry. Imagine projecting a circle of radius L_d around the robot. The intersection of this circle with the path is the **look-ahead point**. The robot simply steers toward this point.

The Math

Using the bicycle kinematic model, the steering angle required to reach the look-ahead point is calculated as:

$$\delta = \arctan(2L \cdot \sin(\alpha) / L_d)$$

- δ = Steering angle
- L = Robot wheelbase
- α = Heading to look-ahead
- L_d = Look-ahead distance

THE ONLY TUNING KNOB

Unlike PID which requires tuning three parameters (Kp, Ki, Kd), Pure Pursuit has only one: the **Look-ahead Distance (L_d)**. This single parameter controls the entire behavior of the robot.

Pure Pursuit: The Trade-off

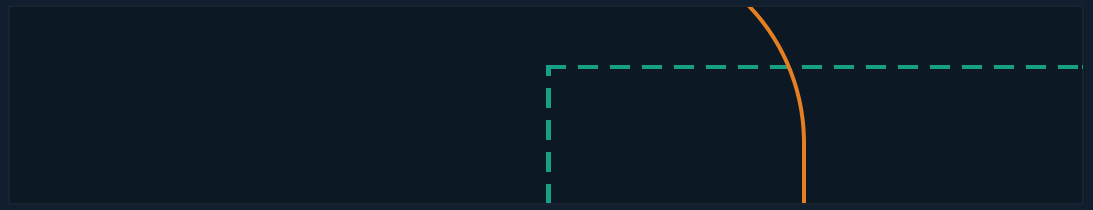
The look-ahead distance (Ld) is the only tuning knob

Small Look-Ahead (Ld)

Provides very tight tracking at low speeds, but causes **severe oscillation and instability** at higher speeds. The robot overcorrects constantly.

Large Look-Ahead (Ld)

Provides very smooth trajectories and stability at high speeds, but **cuts corners** and deviates significantly from the desired path on sharp turns.



THE SOLUTION

Adaptive Look-Ahead

$$Ld = k \cdot v$$

Instead of a fixed distance, scale the look-ahead linearly with the robot's velocity (v).

- **Low speed:** Small Ld for precise maneuvering.
- **High speed:** Large Ld for stability and smoothness.

The tuning parameter becomes **k** (a time constant, usually 0.1 to 0.5 seconds).

Real-World Application

This adaptive Pure Pursuit formulation is the exact algorithm used by early versions of **Tesla Autopilot** and remains the default local planner in the **ROS Navigation Stack**. It is computationally cheap and highly effective.

Stanley Controller: The Winner

Combining heading error and CTE in one elegant equation

$$\delta(t) = \psi(t) + \arctan(k \cdot e(t) / v(t))$$

Heading Error Correction

Directly aligns the robot's wheels with the path's tangent. If the path turns left by 10°, the wheels turn left by 10°.

Cross-Track Error Correction

Pulls the robot back to the path. The correction is proportional to the error (**e**) and inversely proportional to velocity (**v**). At high speeds, it steers less to prevent rolling over.

The DARPA Grand Challenge

Developed by Stanford University's racing team for "Stanley", the autonomous Volkswagen Touareg that won the 2005 DARPA Grand Challenge (a 132-mile off-road race). It proved so robust that it became a standard in the autonomous vehicle industry.

STANLEY VS. PURE PURSUIT

Pure Pursuit	Stanley Controller
✓ Uses a look-ahead point	✓ Uses the front axle position
✓ Very smooth at high speeds	✓ Tracks sharp corners perfectly
× Cuts corners on sharp turns	✓ Explicitly corrects heading
× Ignores heading error directly	× Can be jerky at low speeds

Choosing the Right Controller

A comparative analysis of path tracking algorithms

CONTROLLER	CORE APPROACH	TUNING	KEY ADVANTAGE	KEY WEAKNESS	BEST FOR
PID	Mathematical error correction (P, I, D terms) applied to CTE.	High (3 Params)	Universal, simple to implement, handles steady-state error well.	Ignores path geometry; highly sensitive to speed changes.	Simple AGVs
Pure Pursuit	Geometric intersection of a look-ahead circle with the path.	Low (1 Param)	Very smooth trajectories; naturally handles high speeds.	Cuts corners on sharp turns; can oscillate at low speeds.	Highway AVs
Stanley	Non-linear feedback combining heading error and CTE.	Medium (1-2 Params)	Provably stable; excellent at tracking tight, complex paths.	Math is more complex; requires accurate heading estimation.	Urban AVs

Key Takeaways

The core concepts of path tracking and control

01 **Two Errors to Rule Them All:** Every path tracking controller's goal is to simultaneously minimize Cross-Track Error (CTE) and Heading Error (ψ).

02 **PID is the Baseline:** It corrects CTE using proportional, integral, and derivative terms, but requires careful tuning and ignores path geometry.

03 **Pure Pursuit is Geometric:** It works by projecting a circle ahead of the robot and steering toward the intersection point on the path.

04 **The Look-Ahead Trade-off:** In Pure Pursuit, a small Ld causes oscillation, while a large Ld cuts corners. Adaptive look-ahead (Ld = k·v) solves this.

05 **The Stack is Complete:** Path tracking is the final bridge between the planned waypoints (Lab 5) and the physical wheel motors.

Lab 6 Preview

ACTIVITY 1

PID Path Follower

Implement a PID controller in pure Python to follow a curved path. Tune Kp, Ki, and Kd to achieve a critically damped response.

ACTIVITY 2

Pure Pursuit Controller

Implement the geometric look-ahead math. Compare its tracking performance against your PID controller on the same path.

ACTIVITY 3 (CAPSTONE INTEGRATION)

PyBullet Warehouse Navigation

Attach your Pure Pursuit controller to the **A* path from Lab 5**. Watch the robot autonomously drive the planned route in the 3D warehouse.

LAB 6

AUTONOMY STACK COMPLETE

Questions & Discussion

Ready for the Capstone Project

Dr. Akram Fatayer

a.fatayer@zuj.edu.jo linkedin.com/in/akram-fatayer